Computer Architecture

Project 1: Processor Scheduling, Performance, and Energy Trade-offs

Project report

Name	Ankit Saini
Student ID	242084
GitHub	https://github.com/GITGUD360/P1-Processor-Scheduling
Date	November 2026

Summary

For this project I fixed the six scheduling decisions in the simulator (G1 to G6), checked that the simulator now behaves the way the model contract says, and then used the verified results to choose a processor design for the IoT task.

After my changes `make check` passes (40/40 decision cases, 39/39 short programs, framework tests OK) and `make evaluate` gives 12 out of 12 records marked VERIFIED_FOR_THIS_RUN. Everything in the Studies uses those verified runs at control_scale = 1.

My choices in short:

- Serial workload, 140 ns deadline: **In-order** (118.25 ns, 5.695 nJ per period)
- Four streams, 140 ns deadline: **Scoreboard** (82.00 ns, 5.536 nJ per period)
- Four streams, 70 ns deadline: **Tomasulo**, the only design that makes it (59.75 ns, 5.798 nJ per period)

1. Coding tasks (G1 to G6)

I only changed the marked return expressions in `src/inorder_control.cpp`, `src/scoreboard_control.cpp` and `src/tomasulo_control.cpp`, and added a short comment above each one. Nothing else in the project was touched. I did them in the order the handout suggests: G1, then G2 and G3, then G5 and G6, and G4 last.

Task	Starter	My version
G1	`sources_ready && fu_free && !older_inflight`	`sources_ready && fu_free`
G2	`row_free && destination_tag == 0 && !any_inflight`	`row_free && destination_tag == 0`
G3	`same_register`	`ready_unread && same_register`
G4	`rs_free && fu_free && !any_inflight`	`rs_free`
G5	`false`	`producer_tag != 0`
G6	`true`	`current_tag == result_tag`

Table 1: The six decisions before and after.

G1 (In-order launch). The controller only ever gives G1 the oldest waiting instruction, so starts are already in program order. Real dependences are covered by `sources_ready`, and two instructions fighting over the same unit is covered by `fu_free`. Waiting for all older work to finish just serialized everything, even when the older instruction was on a different unit, so I removed that check.

G2 (Scoreboard issue). Issue should only stop for a structural hazard (row/FU busy) or a WAW hazard (something is already going to write the same destination), not just because other work is in flight.

G3 (Scoreboard WAR). A WAR hazard only matters while an older instruction still needs the old value: it reads the same register and that value is ready but has not been read yet. If the reader is still waiting for its own producer, or it already grabbed the value, the write can go ahead.

G4 (Tomasulo issue). Getting into a reservation station and starting on a unit are separate steps. A free RS is enough to issue; the FU is checked at launch and renaming handles WAW.

G5 (wait or capture). A nonzero RAT tag means a newer value is still coming, so the source keeps the tag and waits. Tag 0 means the register file is up to date, so the value is taken right away.

G6 (register update). Only the producer the RAT still points to may write the register and clear the RAT entry. If a younger instruction renamed it in the meantime, the old result still goes to its waiting consumers but must not overwrite the newer mapping.

2. Validation

Before editing anything I ran `make demo` on the untouched starter and kept the output (screenshot S1, results kept in `results_before/demo`). All three models came out UNVERIFIED there, and none of them met the 140 ns deadline on the serial workload.

After the fixes I ran `make check` and then `make evaluate` on the final code:

Check	Result
Decision cases (G1 to G6)	40 / 40 passed
Framework tests	PASS
Short programs (13 programs x 3 models, all timing, events, registers, energy)	39 / 39 passed
`make check`	CHECK: PASS
`make evaluate`	EVALUATION: PASS, 12 / 12 VERIFIED_FOR_THIS_RUN

Table 2: Validation results for the submitted code.

I worked on Windows 11 using Git Bash, g++ 16.2 (MinGW-w64), GNU Make 4.4.1 and Python 3.12. On Windows the compiler writes `build/sim.exe`, so I used a small wrapper that also copies it to `build/sim` (the name the workflow hashes) and points `python3` at my Python install. I didn't change any project file for this. The results still match the bundled reference signatures, which is why every run comes out verified.

3. Experiments and architecture selection

The rule I used is the one from the handout: first throw out anything that is wrong or misses the deadline, then pick the lowest **period** energy from what is left. Clock is 4 GHz (0.25 ns per cycle), the period is 1,000 ns and sleep power is 0.10 mW.

Study A: Serial workload

(a) Results

Design	Cycles	Time (ns)	Bkg (mW)	E_dyn (nJ)	E_bkg (nJ)	E_task (nJ)	P_avg (mW)	E_period (nJ)	140 ns?
In-order	473	118.25	20	3.242	2.365	5.607	47.42	5.695	Yes
Scoreboard	442	110.50	24	3.591	2.652	6.243	56.49	6.332	Yes
Tomasulo (8 RS)	442	110.50	30	4.432	3.315	7.747	70.11	7.836	Yes

Table 3: iot_serial at control_scale = 1. All three pass the functional oracle and meet the deadline (560 cycles).

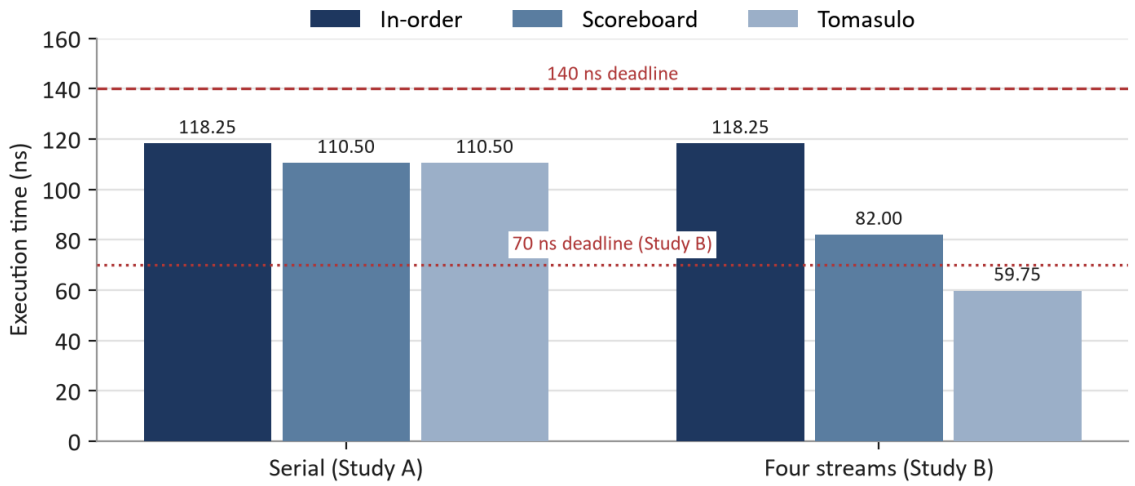


Figure 1: Execution time for both workloads against the 140 ns and 70 ns deadlines.

(b) Recommendation: In-order. All three designs are correct and meet 140 ns, so it comes down to period energy, and In-order has the lowest at 5.695 nJ. That is 0.636 nJ (about 10%) less than Scoreboard and 2.140 nJ (about 27%) less than Tomasulo, and In-order still has 21.75 ns (87 cycles) to spare.

The out-of-order designs are only 31 cycles faster here, and the reason is the F0 chain. Every sample has to wait for the previous FDIV, and the chain FADD, FMUL, FADD, FDIV costs (3+1) + (5+1) + (3+1) + (12+1) = 27 cycles per sample once you count the extra cycle for publishing and waking up. 16 x 27 plus 10 cycles to get the first load going is exactly 442 cycles, which is what both Scoreboard and Tomasulo get. No amount of dynamic scheduling can beat that. In-order loses about 2 cycles per sample because results have to be written back in order. Meanwhile the fancier schedulers burn more background power the whole time (24 and 30 mW against 20 mW) and have more expensive control events. Since sleep is only 0.10 mW, finishing 7.75 ns early saves less than 1 pJ. Being faster than the deadline doesn't earn anything, so In-order is the better choice.

(c) Where the energy difference comes from (In-order vs Tomasulo)

Component	In-order	pJ	Tomasulo	pJ	Diff (pJ)
scheduler_entry_probes	1,668 x 0.12	200.16	3,594 x 0.30	1,078.20	+878.04
tag comparisons	327 x 0.06	19.62	4,200 x 0.06	252.00	+232.38
entry_transitions	463 x 0.25	115.75	463 x 0.75	347.25	+231.50
register_reads	160 x 1.0	160.00	81 x 1.0	81.00	-79.00
register_writes	96 x 1.2	115.20	35 x 1.2	42.00	-73.20
everything else (ops, admission, bus, deliveries)		2,631.50		2,631.50	0
Dynamic total		**3,242.23**		**4,431.95**	**+1,189.72**

Table 4: Energy components from iot_serial_inorder_s1_energy.csv and iot_serial_tomasulo_s1_energy.csv (count x pJ per event).

Working one out by hand, Tomasulo's scheduler probes are:

```
E = 3,594 events x 0.30 pJ/event = 1,078.20 pJ (In-order: 1,668 x 0.12 = 200.16 pJ)
```

Tomasulo probes a lot more because its 8 reservation stations stay busy while instructions sit waiting on the F0 chain, and each probe costs 2.5 times as much. The CDB tag comparators also check every pending source tag and every active RAT entry at the full 4-bit width on each broadcast, matching or not (4,200 bits, so 1,050 tag compares). Tomasulo does save some register file energy, since operands come off the CDB and G6 skips 61 writes to scratch registers that were already renamed, but that is only -152.2

pJ. The probes, tag compares and transitions explain the whole +1,189.72 pJ dynamic gap. On top of that the background power costs 30 x 110.5 - 20 x 118.25 = +950 pJ, and the slightly longer sleep adds 0.775 pJ, which gives the measured period difference of 2,140.5 pJ. Scoreboard vs In-order looks the same, just smaller: probes +215.34 pJ, transitions +115.75 pJ, tags +11.52 pJ and WAR compares +5.76 pJ.

Study B: Four independent streams

(a) Results. For 70 ns I used deadline_70_met, which is the same as checking time_ns against 70 (280 cycles).

Design	Cycles	Time (ns)	E_task (nJ)	E_period (nJ)	140 ns?	70 ns?
In-order	473	118.25	5.593	5.681	Yes	No
Scoreboard	328	82.00	5.444	5.536	Yes	No
Tomasulo (8 RS)	239	59.75	5.704	5.798	Yes	Yes

Table 5: iot_four_streams at control_scale = 1.

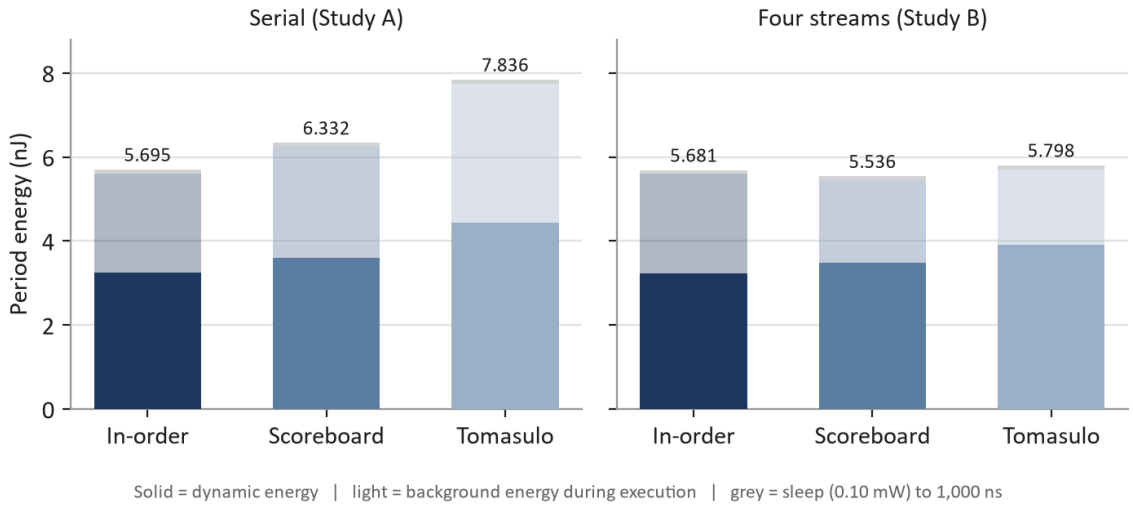


Figure 2: Period energy split into dynamic (dark), background while running (light) and sleep for the rest of the period (grey).

(b) 140 ns: Scoreboard. All three make the deadline, and Scoreboard has the lowest period energy (5.536 nJ, compared with 5.681 nJ for In-order and 5.798 nJ for Tomasulo). With four separate chains, Scoreboard can overlap the streams and finishes in 82 ns instead of 118.25 ns. Running for a shorter time saves about 397 pJ of background energy (24 x 82 = 1,968 pJ against 20 x 118.25 = 2,365 pJ), which is more than its extra 248 pJ of control energy. In-order takes exactly the same 473 cycles as on the serial program. Completed LD and IADD results still have to wait behind each 12-cycle FDIV to write back in order, and only one instruction can be waiting to launch, so it never gets to use the independence between the streams. Tomasulo is the fastest (59.75 ns), but at 30 mW and with heavier control activity it ends up the most expensive at this deadline.

(c) 70 ns: Tomasulo. Only Tomasulo makes it (239 cycles = 59.75 ns, with 41 cycles to spare). Scoreboard (82 ns) and In-order (118.25 ns) miss, so they are out before energy even comes into it. Scoreboard keeps getting stuck at issue on WAW hazards because all four streams reuse F8 to F11, and its rows are held from issue until write. Tomasulo renames those registers, lets instructions into the RS even when the unit is busy (G4) and forwards results over the CDB. It ends up close to the limit set by the single FDIV unit: each divide holds it for 13 cycles (12 + publish), and 16 x 13 = 208 cycles. So I pick Tomasulo at 5.798 nJ per period, which is 0.262 nJ (4.7%) more than Scoreboard. That is the price of the tighter deadline.

Study C: When would I pick something else?

- **Tighter deadline on the serial workload.** In-order stops working below 118.25 ns. Between 110.5 and 118.25 ns I would switch to Scoreboard, which is as fast as Tomasulo but uses 1.504 nJ less per period. Below 110.5 ns nothing works, because 442 cycles is the limit set by the F0 chain. You would need faster or pipelined FP units, or a different algorithm (hypothesis).
- **More independent work.** Going from one chain to four already moves the choice from In-order to Scoreboard (140 ns) or Tomasulo (70 ns). More streams, or code that keeps reusing the same scratch registers, should favour Tomasulo's renaming even more (hypothesis).
- **Cheaper scheduling hardware.** control_scale doesn't change timing, only control energy. In the verified serial runs In-order is cheapest at every scale (Table 6). I also ran a few extra standalone four-stream runs (UNVERIFIED, since there is no reference for them): Tomasulo becomes the cheapest at 140 ns once control_scale is about 0.6 (5.050 vs 5.095 nJ for Scoreboard), but not at 0.7 (5.237 vs 5.205 nJ). Working it out from the event counts, the crossover is around 0.66.
- **Higher sleep power.** A faster design spends more of the period asleep, which saves P_sleep x (time difference). Tomasulo would beat Scoreboard on four streams once sleep power is above about 260.02 pJ / 22.25 ns = 11.7 mW. Scoreboard would only beat In-order on the serial workload above about 82 mW. At 0.10 mW, idle time is basically free.
- **Number of reservation stations.** Extra UNVERIFIED runs: with 4 RS Tomasulo saves 3 mW of background power, but four streams slows to 336 cycles (84 ns, so it misses 70 ns) and costs 6.082 nJ. With 16 RS it gets to 234 cycles (58.5 ns) but costs 6.678 nJ. Even with 4 RS, serial Tomasulo (6.971 nJ) is still worse than In-order. Eight looks like a sensible middle.
- **Things the model leaves out** (hypothesis): cache misses, variable load latency, branches with speculation or pipelined FUs would make out-of-order latency hiding worth more and could favour Tomasulo even with less parallel code.

control_scale	In-order (nJ)	Scoreboard (nJ)	Tomasulo (nJ)
0.5	5.390	5.638	6.314
1	5.695	6.332	7.836
2	6.307	7.718	10.879

Table 6: Verified serial period energy at the three control scales (just for context). The ranking stays the same.

4. Quick questions (Q1 to Q5)

Q1. Workload and requirements

(a) F0 carries the state from one sample to the next. The FDIV of sample k writes F0 and the first FADD of sample k+1 reads it, so it is a loop-carried **RAW (true) dependence**. The reused scratch registers F8 to F11 also add name dependences (WAR and WAW).

(b) One cycle at 4 GHz is 1 / 4 GHz = 0.25 ns, so:

```
cycle budget = 140 ns / 0.25 ns = 140 x 4 = 560 cycles
```

(c) In the four-stream version the samples go round-robin to four separate states, F0 to F3. Back-to-back samples don't depend on each other and each chain only comes back every fourth sample (24 instructions later), so up to four chains can be in progress at once. They still share the functional units, the scratch registers F8 to F11 (WAR/WAW) and the R1 counter. It also computes different results, so it isn't just a faster version of the serial program.

Q2. A scheduling decision: G4 (Tomasulo, load_admission.asm)

I	Instruction	Issue	Start	Done	Write
1	`LD F0, [0]`	1	2	9	10
2	`IADD R1, R2, R3`	2	3	3	4
3	`LD F1, [1]`	3	11	18	19

Table 7: Timing from results/check/load_admission_tomasulo (screenshot S4).

(a) I3 (`LD F1, [1]`) has no register sources (the address is part of the instruction) and nothing else is writing F1, so it isn't waiting on any data. At cycle 3 only two of the eight reservation stations are in use (I1 and I2), so `rs_free` is true. The one MEMORY unit is still busy with I1, which started at cycle 2 and doesn't write until cycle 10, so `fu_free` is false.

(b) With my G4, issuing only needs a free RS, so I3 gets in at cycle 3 even though the memory unit is busy, and waits there. I1 publishes at cycle 10. A unit freed during publication can't be reused in that same cycle, so I3 starts at 11 and writes at 19. The starter version also needed `fu_free` and nothing in flight, which would have kept I3 (and I2 before it) out of the RS until the machine was empty.

(c) File `tests/cases/load_admission.asm`, instruction I3: Issue = 3 while I1 still has the memory unit (Start 2, Write 10), and I3 Start = 11, one cycle after I1 writes.

Q3. Producer tags and register versions

(a) It should wait. Tag 3 means the register file value is old and producer 3 is going to send the value this instruction needs.

(b) When producer 3 broadcasts, every waiting source with tag 3 sees the match, takes the value and clears its tag to 0 (both operand slots are checked). Those instructions are then ready and can launch from the next cycle.

(c) No. The RAT says 7, which means a younger instruction has already renamed that register and owns the newest version. If producer 3 wrote the register and cleared the RAT, it would wipe out the newer mapping and later readers would get the wrong value. So the register is only updated when `current_tag == result_tag` (here 7 is not 3), but the value is still delivered to the tag-3 consumers because they need that older version.

Q4. Time, power and energy

```
T = 400 cycles / 4 GHz = 100 ns
E_bkg = 20 mW x 100 ns = 2,000 pJ
E_task = 3,000 pJ + 2,000 pJ = 5,000 pJ (5.00 nJ)
P_avg = 5,000 pJ / 100 ns = 50 mW
E_period = 5,000 pJ + 0.10 mW x (1,000 - 100) ns = 5,000 + 90 = 5,090 pJ (5.09 nJ)
```

(b) Adding up latencies assumes one thing runs at a time, but different units work in parallel, so the real time depends on the longest dependence chain plus any stalls and the extra cycles for publishing and waking up. For the serial program the latencies add up to 16 x (8+3+5+3+12+1) = 512 cycles, but In-order actually takes 473 and Tomasulo 442 because the loads and IADDs overlap with the FP chain. On four streams Tomasulo only needs 239. It can go wrong the other way too, since the sum ignores stalls.

Q5. Before and after

Design (serial)	Before cycles	Before ns	Before E_period (nJ)	Before 140?	After cycles	After ns	After E_period (nJ)	After 140?
In-order	609	152.25	6.358	No	473	118.25	5.695	Yes
Scoreboard	704	176.00	7.693	No	442	110.50	6.332	Yes
Tomasulo (8 RS)	704	176.00	8.934	No	442	110.50	7.836	Yes

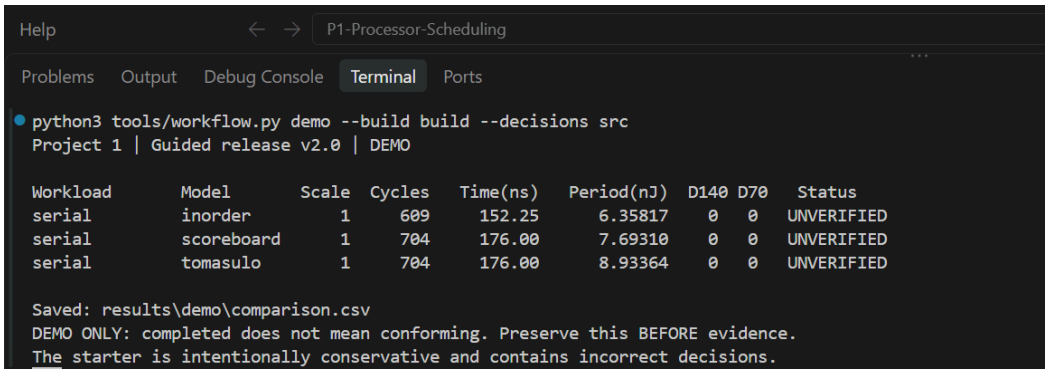
Table 8: Untouched starter (make demo, UNVERIFIED, S1) compared with my final verified runs (make evaluate, S3).

(a) I picked In-order. Before the fix it took 609 cycles (152.25 ns) and 6.358 nJ per period, which misses the deadline. After the fix it takes 473 cycles (118.25 ns) and 5.695 nJ, so 136 fewer cycles and 0.663 nJ (about 10%) less energy, and it now meets 140 ns.

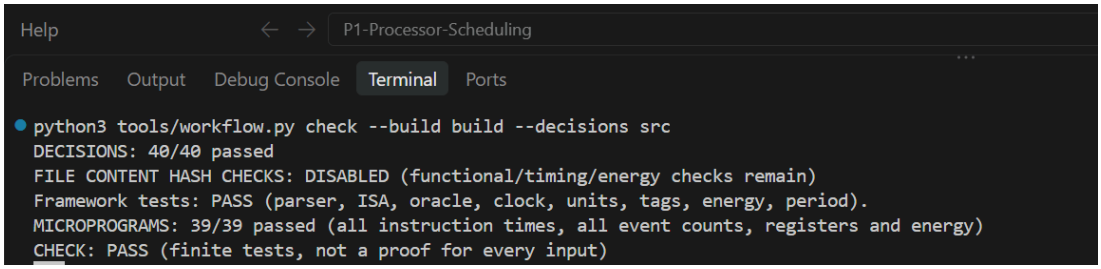
(b) The fix that explains it is G1. The starter only let the head instruction launch if nothing older was still in flight. In the before run, the IADD (I6) couldn't start until the FDIV (I5) published at cycle 37, so it started at 38, and the next LD (I7) started at 40. After the fix, I6 starts at 26 and I7 at 27, overlapping with the 12-cycle FDIV on different units. Each sample now takes 29 cycles instead of 38, and the shorter run saves about 20 mW x 34 ns = 680 pJ of background energy. Scoreboard (G2) and Tomasulo (G4) had the same kind of over-restriction and each dropped by 262 cycles.

5. Screenshots

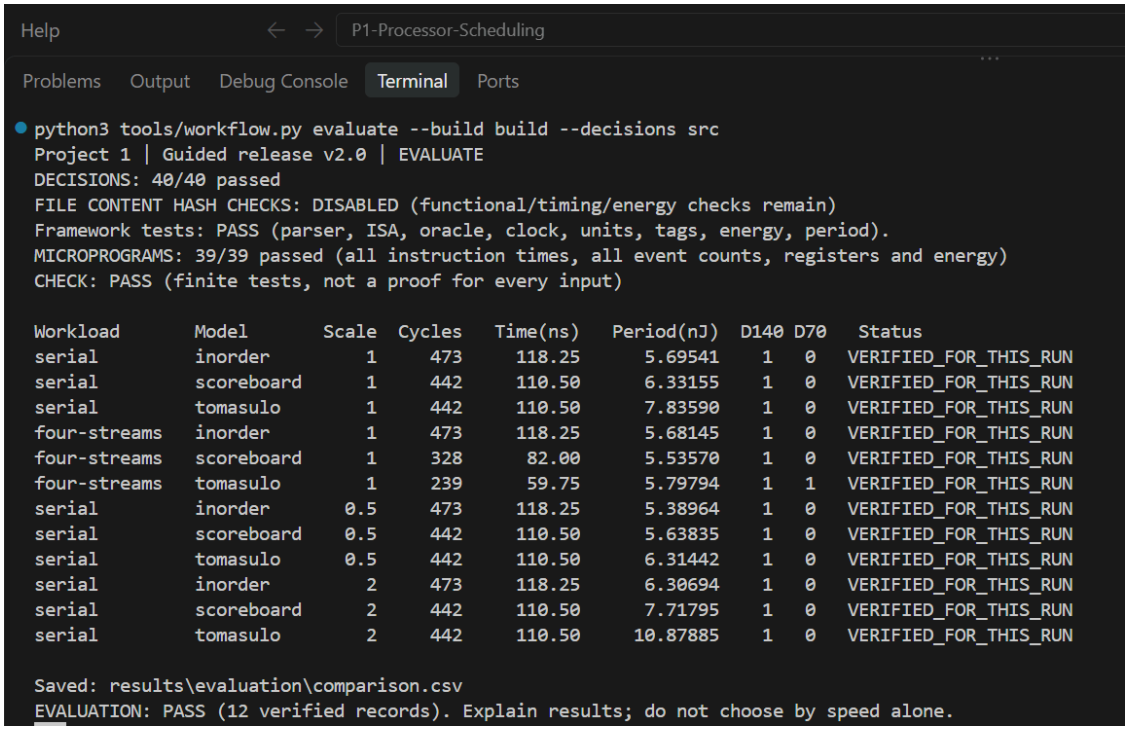
All of these come from my own runs of the project.



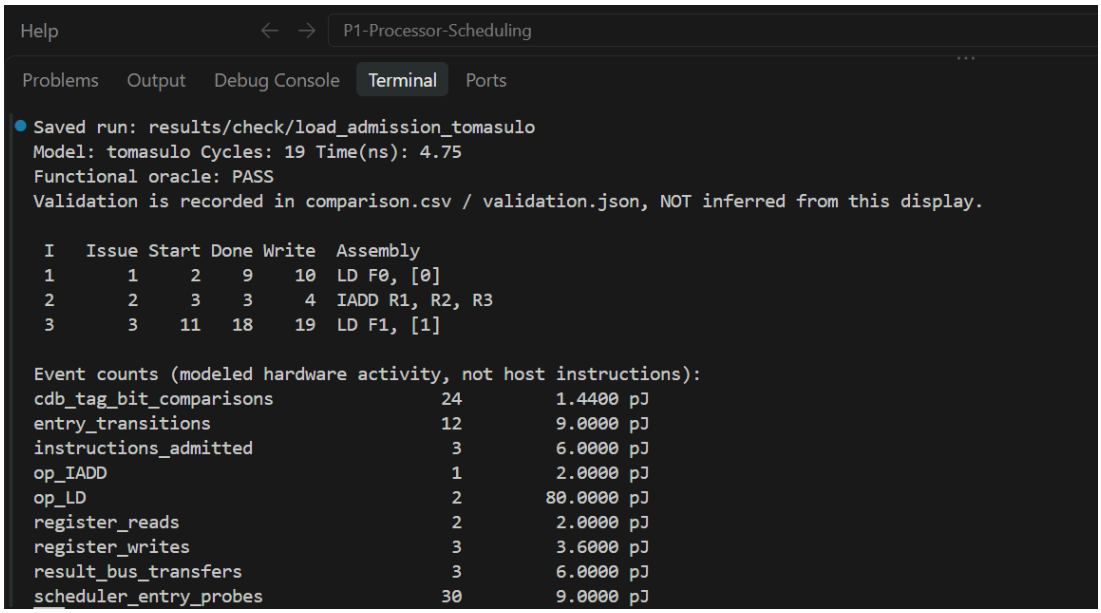
S1. Before any changes: `make demo` on the starter code. All three models are UNVERIFIED.



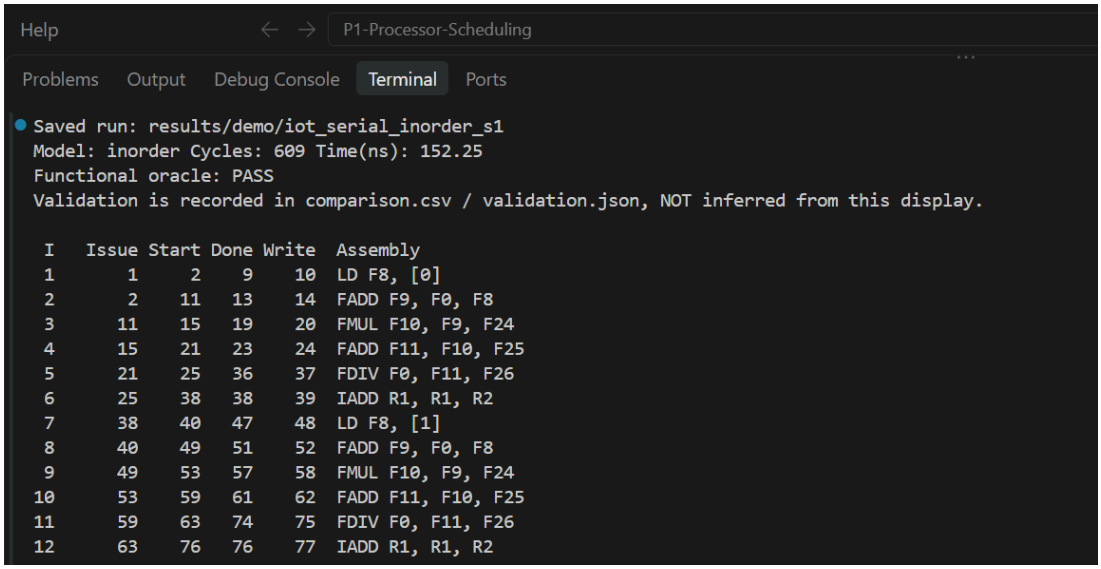
S2. After the fixes: `make check`. 40/40 decisions, framework PASS, 39/39 short programs, CHECK: PASS.



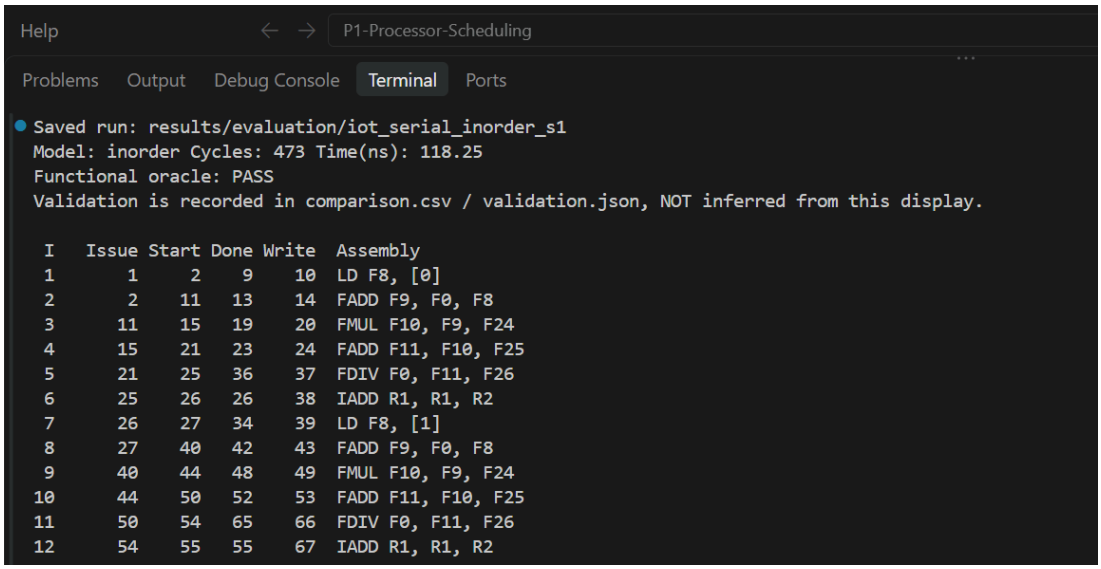
S3. `make evaluate`. All rows VERIFIED_FOR_THIS_RUN (time in ns, period energy in nJ).



S4. Q2 evidence (G4, Tomasulo, load_admission.asm): I3 issues at cycle 3 while I1 has the memory unit, and starts at 11 after I1 writes at 10.



S5a. Q5 before: In-order serial timing. I6 starts at 38, after the FDIV (I5) writes at 37.



S5b. Q5 after: I6 starts at 26 and I7 at 27, overlapping the FDIV (I5, Start 25, Write 37).